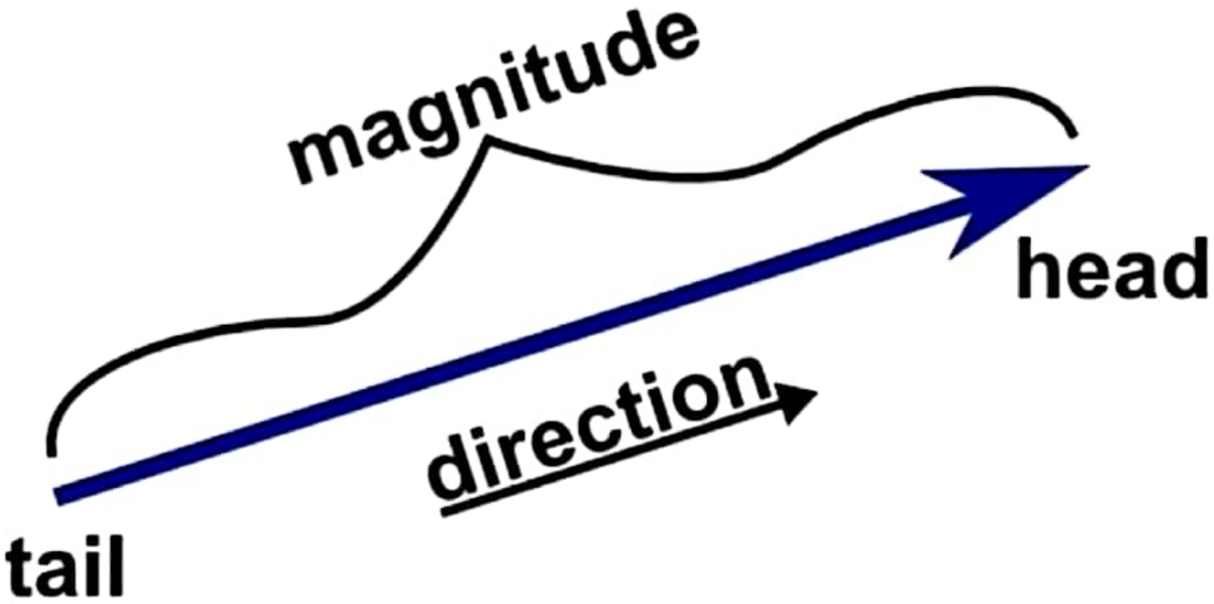
What is a vector?

A vector is how we represent text numerically. Vectors are also at the core of many machine learning approaches, which can often be treated as purely geometrical problems.



A vector is a quantity that has both magnitude and direction. This contrasts with a scalar, which only has a magnitude and possibly a sign (+/-), but no direction.

For example:

- Velocity is a vector because it has both magnitude and direction.
  Example: My velocity is 10 meters per second towards the northeast.
- Speed is a scalar.
  Example: My speed is 10 meters per second, without any associated direction.

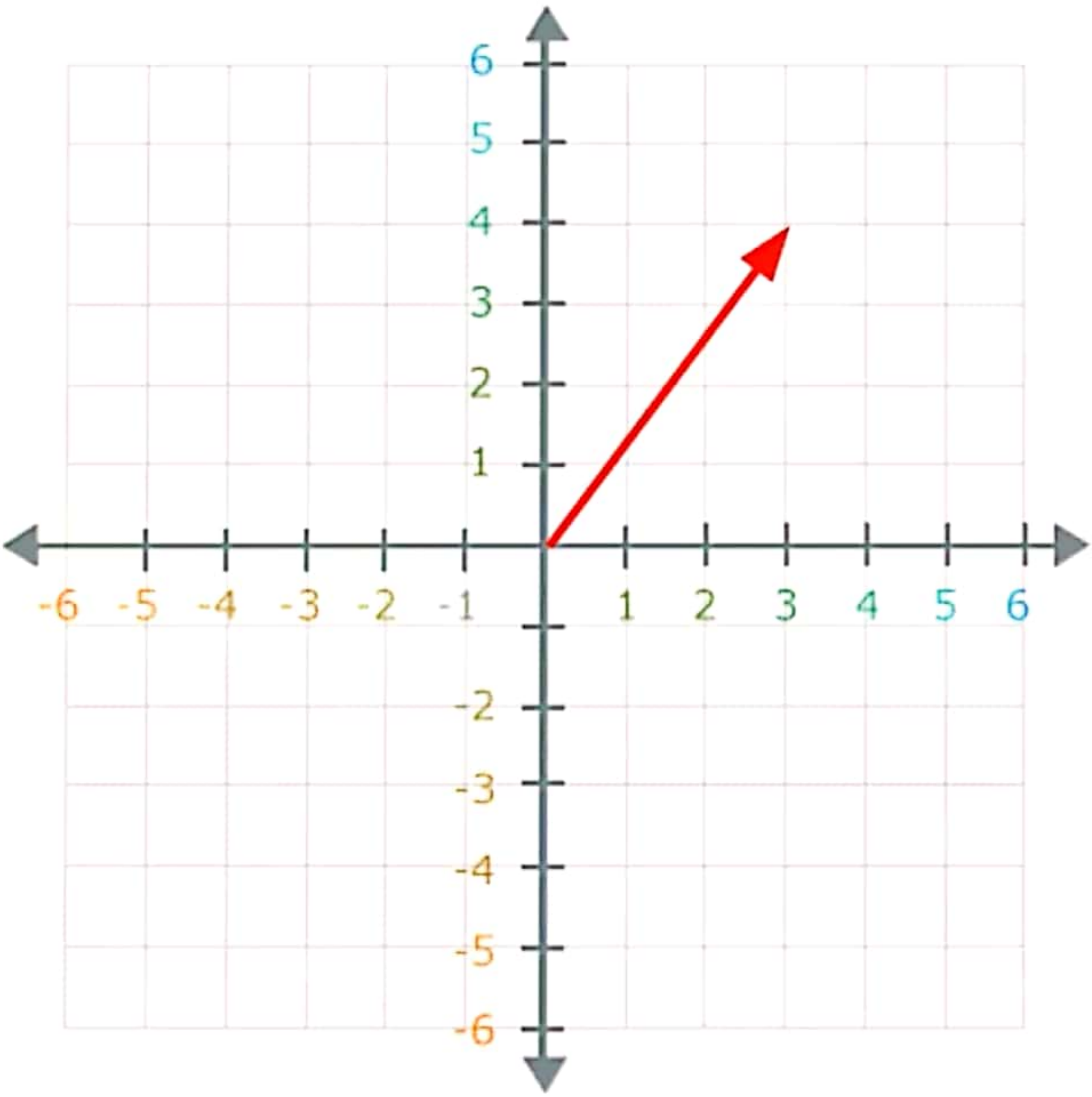
Vectors in High Dimensions

The second way to think about vectors is more useful, especially in machine learning, where we work in very high dimensions.

In high-dimensional spaces:

- Direction becomes unintuitive.
- The number of angles needed to define direction increases with the number of dimensions.

Therefore, we often use the second approach: representing vectors as arrays of scalar values. This representation is generally more practical for computation.



The Bag-of-Words Model (BoW)

A BoW model is one of the more simplistic feature extraction algorithms that you will come across. The name "bag-of-words" comes from the algorithm simply seeking to know the number of times a given word is present within a body of text. The order or context of the words is not analyzed here. Similarly, if we have a bag filled with six pencils, eight pens, and four notebooks, the algorithm merely cares about recording the number of each of these objects, not the order in which they are found, or their orientation.

Here, I have defined a sample bag-of-words function.

```
def bag_of_words(text):
    _bag_of_words = [collections.Counter(re.findall(r'\w+',
    word)) for word in text]
    bag_of_words = sum(_bag_of_words, collections.Counter())
    return bag_of_words

sample_word_tokens_bow = bag_of_words(text=sample_word_tokens)
print(sample_word_tokens_bow)
```

When we execute the preceding code, we get the following output:

```
Counter({'philosophy': 2, 'program': 1, 'chances': 1, 'years': 1,
'states': 1, 'born': 1, 'towards': 1, 'canada': 1, 'huge': 1,
'united': 1, 'goal': 1, 'working': 1, 'decision': 1,
'currently': 1, 'confident': 1, 'going': 1, '4': 1,
'difficult': 1, 'good': 1, 'degree': 1, 'get': 1, 'becoming': 1,
'phd': 1, 'ontario': 1, 'fan': 1, 'student': 1, 'improve': 1,
'professor': 1, 'enrolled': 1, 'alabama': 1, 'university': 1})
```

This is an example of a BoW model when presented as a dictionary. Obviously, this is not a suitable input format for a machine learning algorithm. This brings me to discuss the myriad of text preprocessing

functions available in the scikit-learn library, which is a Python library that all data scientists and machine learning engineers should be familiar with. For those who are new to it, this library provides implementations of machine learning algorithms, as well as several data preprocessing algorithms. Although we won't walk through much of this package, the text preprocessing functions are extremely useful.

CountVectorizer

Let's start by walking through the BoW equivalent—CountVectorizer, an implementation of bag-of-words in which we code text data as a representation of features/words. The values of each of these features represent the occurrence counts of words across all documents. If you recall, we defined a `sample_sent_tokens` variable, which we will analyze. We define a `bow_sklearn()` function beneath where we preprocess our data. The function is defined as follows:

```
from sklearn.feature_extraction.text import CountVectorizer
def bow_sklearn(text=sample_sent_tokens):
    c = CountVectorizer(stop_words='english',
    token_pattern=r'\w+')
    converted_data = c.fit_transform(text).todense()
    print(converted_data.shape)
    return converted_data, c.get_feature_names()
```

To provide context, in this example, we are assuming that each sentence is an individual document, and we are creating a feature set in which each feature is an individual token. When we instantiate `CountVectorizer()`, we set two parameters: `stop_words`, and `token_pattern`. These two arguments are the embedded methods in the feature extraction that remove stop words and grammatical tokens. The `fit_transform()` attribute expects to receive a list, an array, or a similar object of iterable string objects. We assign the `bow_data` and `feature_names` variables to the data that the

`bow_sklearn()` returns, respectively. Our converted data set is a 6 × 50 matrix, which means that we have six sentences, all of which have 50 features. Observe our data set and feature names, respectively, in the following outputs:

```
[[0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0]
 [0 0 1 1 0 0 0 0 0 0 0 1 0 0 0 1 0 1 0 0 0 0 1 0 1 0 0 0]
 [0 0 0 0 1 0 0 0 1 0 0 0 0 1 0 0 1 0 0 2 1 0 0 0 0 0 0 0]
 [1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1]
 [0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0]
 [0 0 0 0 0 1 0 1 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0]]
```

```
[u'4', u'alabama', u'born', u'canada', u'chances',
u'confident', u'currently', u'decision', u'degree',
u'difficult', u'enrolled', u'fan', u'goal', u'going', u'good',
u'huge', u'improve', u'ontario', u'phd', u'philosophy',
u'professor', u'program', u'states', u'student', u'united',
u'university', u'working', u'years']
```

To extrapolate this example to a larger number of documents, and ostensibly larger vocabulary sizes, our matrices for preprocessed text data tends to have a large number of features, sometimes well over 1000. How to evaluate these features effectively is the machine learning challenge we seek to solve. You typically want to use the bag-of-words feature extraction technique for document classification. Why is this the case? We assume that documents of certain classifications contain certain words. For example, we expect a document referencing political science to perhaps feature jargon such as *dialectical materialism* or *free market capitalism*; whereas a document that is referring to classical music will have terms such as *crescendo*, *diminuendo*, and so forth. In these instances of document classification, the location of the word itself is not terribly important. It's important to know what portion of the vocabulary is present in one class of document vs. another.

Next, let's look at our first example problem in the code in the `text_classifiction_demo.py` file.

Tokenization and Stop Words

When you are working with raw text data, particularly if it uses a web crawler to pull information from a website, for example, you must assume that not all of the text will be useful to extract features from. In fact, it is likely that more noise will be introduced to the data set and make the training of a given machine learning model less effective. As such, I suggest that you perform preliminary steps. Let's walk through these steps using the following sample text.

```
sample_text = "'I am a student from the University of Alabama. I
was born in Ontario, Canada and I am a huge fan of the United
States. I am going to get a degree in Philosophy to improve
my chances of becoming a Philosophy professor. I have been
working towards this goal for 4 years. I am currently enrolled
in a PhD program. It is very difficult, but I am confident that
it will be a good decision"'
```

When the sample_text variable prints, there is the following output:

```
'I am a student from the University of Alabama. I
was born in Ontario, Canada and I am a huge fan of the United
States. I am going to get a degree in Philosophy to improve my
chances of becoming a Philosophy professor. I have been working
towards this goal for 4 years. I am currently enrolled in a PhD
program. It is very difficult, but I am confident that it will
be a good decision'
```

You should observe that the computer reads bodies of text, even if punctuated, as single string objects. Because of this, we need to find a way to separate this single body of text so that the computer evaluates each word as an individual string object. This brings us to the concept of *word tokenization*, which is simply the process of separating a single string object, usually a body of text of varying length, into individual tokens that represent words or characters that we would like to evaluate further. Although you can find ways to implement this from scratch, for brevity's sake, I suggest that you utilize the Natural Language Toolkit (NLTK) module.

NLTK allows you to use some of the more basic NLP functionalities, as well as pretrained models for different tasks. It is my goal to allow you to train your own models, so we will not be working with any of the pretrained models in NLTK. However, you should read through the NLTK module documentation to become familiar with certain functions and algorithms that expedite text preprocessing. Relating back to our example, let's tokenize the sample data via the following code:

```
from nltk.tokenize import word_tokenize, sent_tokenize
sample_word_tokens = word_tokenize(sample_text)
sample_sent_tokens = sent_tokenize(sample_text)
```

When you print the `sample_word_tokens` variable, you should observe the following:

```
['I', 'am', 'a', 'student', 'from', 'the', 'University', 'of',
'Alabama', '.', 'I', 'was', 'born', 'in', 'Ontario', ',',
'Canada', 'and', 'I', 'am', 'a', 'huge', 'fan', 'of', 'the',
'United', 'States', '.', 'I', 'am', 'going', 'to', 'get', 'a',
'degree', 'in', 'Philosophy', 'to', 'improve', 'my', 'chances',
'of', 'becoming', 'a', 'Philosophy', 'professor', '.', 'I',
'have', 'been', 'working', 'towards', 'this', 'goal', 'for',
'4', 'years', '.', 'I', 'am', 'currently', 'enrolled', 'in',
'a', 'PhD', 'program', '.', 'It', 'is', 'very', 'difficult',
',', 'but', 'I', 'am', 'confident', 'that', 'it', 'will', 'be',
'a', 'good', 'decision']
```

You will also observe that we have defined another tokenized object, `sample_sent_tokens`. The difference between `word_tokenize()` and `sent_tokenize()` is simply that the latter tokenizes text by sentence delimiters. This is observed in the following output:

```
['I am a student from the University of Alabama.', 'I \nwas
born in Ontario, Canada and I am a huge fan of the United
States.', 'I am going to get a degree in Philosophy to improve
my chances of \nbecoming a Philosophy professor.', 'I have
been working towards this goal\nfor 4 years.', 'I am currently
enrolled in a PhD program.', 'It is very difficult, \nbut I am
confident that it will be a good decision']
```

Now we have individual tokens that we can preprocess! From this step forward, we can clean out some of the junk text that we would not want to extract features from. Typically, the first thing we want to get rid of are *stop words*, which are usually defined as very common words in a given language. Most often, lists of stop words that we build or utilize in software packages include *function words*, which are words that express a grammatical relationship (rather than having an intrinsic meaning). Examples of function words include *the*, *and*, *for*, and *of*.

In this example, we use the list of stop words from the NLTK package.

```
[u'i', u'me', u'my', u'myself', u'we', u'our', u'ours',
u'ourselves', u'you', u"you're", u"you've", u"you'll",
u"you'd", u'your', u'yours', u'yourself', u'yourselves',
u'he', u'him', u'his', u'himself', u'she', u"she's", u'her',
u'hers', u'herself', u'it', u"it's", u'its', u'itself',
u'they', u'them', u'their', u'theirs', u'themselves', u'what',
u'which', u'who', u'whom', u'this', u'that', u"that'll",
u'these', u'those', u'am', u'is', u'are', u'was', u'were',
```

```
u'be', u'been', u'being', u'have', u'has', u'had', u'having',
u'do', u'does', u'did', u'doing', u'a', u'an', u'the', u'and',
u'but', u'if', u'or', u'because', u'as', u'until', u'while',
u'of', u'at', u'by', u'for', u'with', u'about', u'against',
u'between', u'into', u'through', u'during', u'before',
u'after', u'above', u'below', u'to', u'from', u'up', u'down',
u'in', u'out', u'on', u'off', u'over', u'under', u'again',
u'further', u'then', u'once', u'here', u'there', u'when',
u'where', u'why', u'how', u'all', u'any', u'both', u'each',
u'few', u'more', u'most', u'other', u'some', u'such', u'no',
u'nor', u'not', u'only', u'own', u'same', u'so', u'than',
u'too', u'very', u's', u't', u'can', u'will', u'just', u'don',
u"don't", u'should', u"should've", u'now', u'd', u'll',
u'm', u'o', u're', u've', u'y', u'ain', u'aren', u"aren't",
u'couldn', u"couldn't", u'didn', u"didn't", u'doesn',
u"doesn't", u'hadn', u"hadn't", u'hasn', u"hasn't", u'haven',
u"haven't", u'isn', u"isn't", u'ma', u'mightn', u"mightn't",
u'mustn', u"mustn't", u'needn', u"needn't", u'shan', u"shan't",
u'shouldn', u"shouldn't", u'wasn', u"wasn't", u'weren',
u"weren't", u'won', u"won't", u'wouldn', u"wouldn't"]
```

All of these words are lowercase by default. You should be aware that string objects must exactly match to return a true Boolean variable when comparing two individual strings. To put this more plainly, if we were to execute the code *"you" == "YOU"*, the Python interpreter returns false. The specific instance in which this affects our example can be observed by executing the `mistake()` and `advised_preprocessing()` functions, respectively. Observe the following outputs:

```
['I', 'student', 'University', 'Alabama', '.', 'I', 'born',
'Ontario', ',', 'Canada', 'I', 'huge', 'fan', 'United',
'States', '.', 'I', 'going', 'get', 'degree', 'Philosophy',
'improve', 'chances', 'becoming', 'Philosophy', 'professor',
'.', 'I', 'working', 'towards', 'goal', '4', 'years', '.',
'I', 'currently', 'enrolled', 'PhD', 'program', '.', 'It',
'difficult', ',', 'I', 'confident', 'good', 'decision']
```

```
['student', 'University', 'Alabama', '.', 'born', 'Ontario',
',', 'Canada', 'huge', 'fan', 'United', 'States', '.',
'going', 'get', 'degree', 'Philosophy', 'improve', 'chances',
'becoming', 'Philosophy', 'professor', '.', 'working',
'towards', 'goal', '4', 'years', '.', 'currently', 'enrolled',
'PhD', 'program', '.', 'difficult', ',', 'confident', 'good',
'decision']
```

As you can see, the `mistake()` function does not catch the uppercase "I" characters, meaning that there are several stop words still in the text. This is solved by uppercasing all the stop words and then evaluating whether each uppercase word in the sample text was in the `stop_words` list. This is exemplified with the following two lines of code:

```
stop_words = [word.upper() for word in stopwords.
words('english')]
word_tokens = [word for word in sample_word_tokens if word.
upper() not in stop_words]
```

Although embedded methods in feature extraction algorithms likely account for this case, you should be aware that strings must match exactly, and you must account for this when preprocessing manually.

That said, there is junk data that you should be aware of—specifically, the grammatical characters. You will be relieved to hear that the `word_tokenize()` function also categorizes colons and semicolons as individual word tokens, but you still have to get rid of them. Thankfully, NLTK contains another tokenizer worth knowing about, which is defined and utilized in the following code:

```
from nltk.tokenize import RegexpTokenizer
tokenizer = RegexpTokenizer(r'\w+')
sample_word_tokens = tokenizer.tokenize(str(sample_word_
tokens))
sample_word_tokens = [word.lower() for word in sample_word_
tokens]
```

When we print the `sample_word_tokens` variable, we get the following output:

```
['student', 'university', 'alabama', 'born', 'ontario',
'canada', 'huge', 'fan', 'united', 'states', 'going', 'get',
'degree', 'philosophy', 'improve', 'chances', 'becoming',
'philosophy', 'professor', 'working', 'towards', 'goal',
'4', 'years', 'currently', 'enrolled', 'phd', 'program',
'difficult', 'confident', 'good', 'decision']
```

In the course of this example, we have reached the final step! We have removed all the standard stop words, as well as all grammatical tokens. This is an example of a document that is ready for feature extraction, whereupon some additional preprocessing may occur.

Next, I'll discuss some of the various feature extraction algorithms. And let's work on denser sample data alongside a preprocessed example paragraph.

Recipe 2-7. Stemming

This recipe discusses stemming. Stemming is the process of extracting a root word. For example, *fish*, *fishes*, and *fishing* are stemmed into *fish*.

Problem

You want to do stemming.

Solution

The simplest way is to use NLTK or the TextBlob library.

Recipe 2-8. Lemmatizing

This recipe discusses lemmatization, the process of extracting a root word by considering the vocabulary. For example, *good, better*, or *best* is lemmatized into *good*.

The part of speech of a word is determined in lemmatization. It returns the dictionary form of a word, which must be valid. While stemming just extracts the root word.

- Lemmatization handles matching *car* to *cars* along with matching *car* to *automobile*.
- Stemming handles matching *car* to *cars*.

Lemmatization can get better results.

- The stemmed form of *leafs* is *leaf*.
- The stemmed form of *leaves* is *leav*.
- The lemmatized form of *leafs* is *leaf*.
- The lemmatized form of *leaves* is *leaf*.

Problem

You want to perform lemmatization.

Solution

The simplest way is to use NLTK or the TextBlob library.

Recipe 3-1. Converting Text to Features Using One-Hot Encoding

One-hot encoding is the traditional method used in feature engineering. Anyone who knows the basics of machine learning has come across one-hot encoding. It is the process of converting categorical variables into features or columns and coding one or zero for that particular category. The same logic is used here, and the number of features is the number of total tokens present in the corpus.

Problem

You want to convert text to a feature using one-hot encoding.

Solution

One-hot encoding converts characters or words into binary numbers, as shown next.

	I	love	NLP	is	Future
I love NLP	1	1	1	0	0
NLP is future	0	0	1	1	1

How It Works

There are many functions to generate one-hot encoding features. Let's take one function and discuss it in depth.

Step 1-1. Store the text in a variable

The following shows a single line.

```
Text = "I am learning NLP"
```

Step 1-2. Execute a function on the text data

The following is a function from the pandas library to convert text into a feature.

```
# Importing the library

import pandas as pd

# Generating the features

pd.get_dummies(Text.split())

    Result :

    I  NLP  am  learning
0   1    0   0         0
1   0    0   1         0
2   0    0   0         1
3   0    1   0         0
```

The output has four features since the number of distinct words present in the input was 4.

Recipe 3-2. Converting Text to Features Using a Count Vectorizer

The approach used in Recipe 3-1 has a disadvantage. It does not consider the frequency of a word. If a particular word appears multiple times, there is a chance of missing information if it is not included in the analysis. A *count vectorizer* solves that problem. This recipe covers another method for converting text to a feature: the count vectorizer.

Problem

How do you convert text to a feature using a count vectorizer?

Solution

A count vectorizer is similar to one-hot encoding, but instead of checking whether a particular word is present or not, it counts the words that are present in the document.

In the following example, the words I and NLP occur twice in the first document.

	I	love	NLP	is	future	will	learn	In	2month
I love NLP and I will learn NLP in 2 months	2	1	2	0	0	1	1	1	1
NLP is future	0	0	1	1	1	0	0	0	0

How It Works

sklearn has a feature extraction function that extracts features out of text. Let's look at how to execute this. The following imports the CountVectorizer function from sklearn.

```
#importing the function

from sklearn.feature_extraction.text import CountVectorizer

# Text

text = ["I love NLP and I will learn NLP in 2month "]

# create the transform

vectorizer = CountVectorizer()

# tokenizing

vectorizer.fit(text)

# encode document

vector = vectorizer.transform(text)

# summarize & generating output

print(vectorizer.vocabulary_)
print(vector.toarray())
```



```
Result:

{'love': 4, 'nlp': 5, 'and': 1, 'will': 6, 'learn': 3, 'in': 2, '2month': 0}
[[1 1 1 1 1 2 1]]
```

The fifth token, nlp, appears twice in the document

Recipe 3-6. Converting Text to Features Using TF-IDF

The above-mentioned text-to-feature methods have a few drawbacks, hence the introduction of TF-IDF. The following are some of the disadvantages.

- Let's say a particular word appears in all the corpus documents. It achieves higher importance in our previous methods, but that may not be relevant to your case.
- TF-IDF reflects on how important a word is to a document in a collection and hence normalizes words that frequently appear in all the documents.

Problem

You want to convert text to features using TF-IDF.

Solution

Term frequency (TF) is the ratio of the count of a particular word present in a sentence to the total count of words in the same sentence. TF captures the importance of the word irrespective of the length of the document. For example, a word with a frequency of 3 in a sentence with 10 words is different from when the word length of the sentence is 100 words. It should have more importance in the first scenario, which is what TF does. **TF**(t) = (Number of times term t appears in a document) / (Total number of terms in the document).

Inverse document frequency (IDF) is a log of the ratio of the total number of rows to the number of rows in a particular document in which a word is present. IDF = log(N/n), where N is the total number of rows, and n is the number of rows in which the word was present.

IDF measures the rareness of a term. Words like *a* and *the* show up in all the corpus documents, but rare words are not in all documents. So, if a word appears in almost all the documents, that word is of no use since it does not help with classification or information retrieval. IDF nullifies this problem.

TF-IDF is the simple product of TF and IDF that addresses both drawbacks, making predictions and information retrieval relevant.

$$\text{TF-IDF} = \text{TF} * \text{IDF}$$

How It Works

Follow the steps in this section.

Step 6-1. Read the text data

The following is a familiar phrase.

```
Text = ["The quick brown fox jumped over the lazy dog.",
"The dog.",
"The fox"]
```

Step 6-2. Create the features

Execute the following code on the text data.

```
#Import TfidfVectorizer

from sklearn.feature_extraction.text import TfidfVectorizer

#Create the transform

vectorizer = TfidfVectorizer()

#Tokenize and build vocab

vectorizer.fit(Text)
```

```
#Summarize

print(vectorizer.vocabulary_)
print(vectorizer.idf_)
```

This is the result.

```
Text = ["The quick brown fox jumped over the lazy dog.",
"The dog.",
"The fox"]

{'the': 7, 'quick': 6, 'brown': 0, 'fox': 2, 'jumped': 3, 'over': 5,
 'lazy': 4, 'dog': 1}
[ 1.69314718  1.28768207  1.28768207  1.69314718   1.69314718
  1.69314718  1.69314718  1.    ]
```

Observe that the appears in all three documents, so it does not add much value. The vector value is 1, which is less than all the other tokens.

All the methods or techniques you have looked at so far are based on frequency. They are called frequency-based embeddings or features. The next recipe looks at prediction-based embeddings, typically called word embeddings.

🔹 Concept: Vector Similarity

In NLP, Machine Learning, and Data Science, we often represent **words, sentences, or documents** as **vectors** (numerical representations).
To measure how **similar** two items are, we compare their vectors using a **similarity measure**.

🔸 Common Methods to Measure Vector Similarity

1. **Cosine Similarity** ✅ *(most popular)*
2. **Euclidean Distance**
3. **Dot Product**
4. **Manhattan Distance**

We'll focus on **Cosine Similarity**, since it's the **most widely used** in NLP and vector analysis.

🧮 1️⃣ Cosine Similarity

Formula:

$$\text{Cosine Similarity} = \frac{\vec{A} \cdot \vec{B}}{||\vec{A}|| \, ||\vec{B}||}$$

Where:

- $\vec{A} \cdot \vec{B}$ = dot product of vectors A and B
- $||\vec{A}||$ = magnitude (length) of vector A
- $||\vec{B}||$ = magnitude (length) of vector B